

Certificate

This is to certify that Mr./Ms. _____

University Exam Seat Number _____ has successfully completed the assignments for the Lab Course I (USCSP-357 Practical based on USCS-351) during the Academic Year _____ and has scored _____ Marks out of 15.

Teacher In charge

Head
Department of Computer Science

Internal Examiner

External Examiner

Index

Sr. No.	Contents
1.	Introduction
2.	Assignment Completion Sheet
3.	Operations on processes
4.	Simulation of Operating System Shell and its working
5.	Simulation of CPU Scheduling Algorithms
6.	Simulation of demand paging using memory page replacement algorithms

Introduction About the workbook

This workbook is intended to be used by T. Y. B. Sc. (Computer Science) students for the Practical Course based on USCSP-351 (Practical Based on USCS-351, Operating Systems-I) in Semester V.

Operating System is an important core subject of computer science curriculum, and hands-on laboratory experience is critical to the understanding of theoretical concepts studied as part of this course. Study of any programming language is incomplete without hands on experience of implementing solutions using programming paradigms and verifying them in the lab. This workbook provides rich set of problems covering the basic algorithms as well as numerous computing problems demonstrating the applicability and importance of various data structures and related algorithms.

About the Work Book Objectives –

1. Defining clearly the scope of the course
2. Bringing uniformity in the way course is conducted across different Colleges.
3. Continuous assessment of the students.
4. Bring variation and variety in experiments carried out by different students in a batch
5. Providing ready references for students while working in the lab.
6. Catering to the need of slow paced as well as fast paced learners

How to use this book?

This book is mandatory for the completion of the laboratory course. It is a Measure of the performance of the student in the laboratory for the entire duration of the course.

The Operating Systems-I practical syllabus is divided into Four assignments. Each assignment has problems divided into three sets A, B and C.

Instructions to the students:

Please read the following instructions carefully and follow them

- Students are expected to carry workbook during every practical.
- Students should prepare oneself beforehand for the Assignment by reading the relevant material.
- Teacher/Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.
- **Students will be assessed for each exercise on a scale from 0 to 5**

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

Instruction to the Practical In-Charge:

- Explain the assignment and related concepts in around ten minutes using white board if required or by demonstrating the software.
- Choose appropriate problems to be solved by students. Set A is mandatory. Choose problems from set B depending on time availability. Discuss set C with students and encourage them to solve the problems by spending additional time in lab or at home.
- Make sure that students follow the instruction as given above.
- You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- The value should also be entered on assignment completion page of the respective Lab course.

Instructions to the Lab administrator and Exam guidelines:

- You have to ensure appropriate hardware and software is made available to each student
- Do not provide Internet facility in Computer Lab while examination
- Do not provide pen drive facility in Computer Lab while examination.
- The operating system and software requirements are as given below:
 - Operating system: Linux
 - Editor: Any Linux based editor like vi, gedit etc.
 - Compiler: cc or gcc

Assignment Completion Sheet

Sr. No.	Assignment Name	Marks (out 5)	Signature
1	Operations on processes		
2	Simulation of Operating System Shell and its working		
3	Simulation of CPU Scheduling Algorithms		
4	Simulation of demand paging using memory page replacement algorithms		
	Conduct Quiz at the time of Submission		
Total out 20			
a. Out of 10 for practical assignments			
b. Out of 5 for quiz at the time of submission			
c. Total (Out of 15) (a + b)			

Teacher In charge

Assignment No.1: Operations on Processes

Process: A process is basically a program in execution. We write our computer programs in a text file, during execution it is loaded in computer's memory and then it becomes a process. A process performs all the tasks mentioned in the program. A process can be divided into three sections - stack, text and data.

Stack: The process Stack contains the temporary data such as method/function parameters, return address and local variables.

Text: This includes all instructions specified in the program.

Data: This section contains the global and static variables.

In Linux operating system, new processes are created through `fork()` system call.

Fork() System Call:

System call `fork()` is used to create new processes. It takes no arguments and returns a process ID. The newly created process is called child process and the caller process is called as parent process. After a new child process is created, both parent and child processes will execute the next instruction following the `fork()` system call. Therefore, we have to distinguish the parent from the child. This can be done by testing the returned value of `fork()`:

`fork()` returns a zero to the newly created child process and returns a positive value (processID of the child process) to the parent.

If `fork()` returns a negative value, the creation of a child process was unsuccessful.

The returned process ID is of type `pid_t` defined in `sys/types.h`. Normally, the process ID is an integer. Moreover, a process can use function `getpid()` to retrieve the process ID assigned to this process.

Let us take the following example:

```
int main()
{ printf("Before Forking\n");
  fork();
  printf("After Forking\n");
  return 0;
}
```

If the call to `fork()` is executed successfully, Linux will

- Make two identical copies of address spaces, one for the parent and the other for the child.
- Both processes will start their execution at the next statement following the `fork()` call.

Output of above program:

Before Forking

After Forking

After Forking

Here printf() statement after fork() system call is executed by parent as well as child process. Both processes start their execution right after the system call fork(). Since both processes have identical but separate address spaces, those variables initialized before the fork() call have the same values in both address spaces. Since every process has its own address space, any modifications will be independent of the others. In other words, if the parent changes the value of its variable, the modification will only affect the variable in the child process's address space. Other address spaces created by fork() calls will not be affected even though they have identical variable names.

Consider one simpler example that distinguishes the parent from the child.

```
#include <stdio.h>
#include <sys/types.h>
void ChildProcess(); /* child process prototype */
void ParentProcess(); /* parent process prototype */
int main()
{ pid_t pid;
  pid = fork();
  if (pid == 0)
    ChildProcess();
  else
    ParentProcess();
  return 0;
}
void ChildProcess()
{ printf("I am child process..");
}
void ParentProcess()
{ printf("I am parent process..");
}
```

exec() system call

The exec() family of functions **replaces** the current process image with a new process image. It loads the program into the current process space and runs it from the entry point.

The current process is just turned into a new process and hence the process id PID is not changed, this is because we are not creating a new process we are just replacing a process with another process in exec.

The exec() family consists of following functions,

execl(): 'l' is for the command line arguments passed as list to the function.

```
int execl(const char *path, const char *arg, ...);
```

execlp(): 'p' is the path environment variable which helps to find the file passed as an argument to be loaded into process.

```
int execlp(const char *file, const char *arg, ...);
```

execle(): 'e' stands for an array of pointers that points to environment variables and is passed explicitly to the newly loaded process as parameters along with command line arguments passed as list

```
int execle(const char *path, const char *arg, ..., char * const envp[]);
```

execv(): 'v' is for the command line arguments passed as an array of pointers to the function.

```
int execv(const char *path, char *const argv[]);
```

execlp() System Call:

execlp() system call is used after a fork() call by one of the two processes to replace the processes memory space with a new program. This call loads a binary file into memory and starts its execution. So two processes can be easily communicates with each other.

```
#include<sys/types.h
>#include<stdio.h>
#include<unistd.h>
int main()
{
    int pid;
    pid = fork(); /* fork a child process */
    if (pid< 0) /* error occurred */
    { fprintf(stderr, "Fork Failed");
    return 1;
    }
    else if (pid == 0) /* child process */
        execlp("/bin/wc", "wc", NULL);
    else /* parent process */
```

```

        { wait(NULL); /* parent will wait for the child to complete */
          printf("Child Completed");
        }
    return 0;
}

```

fork() vs exec()

- Fork() starts a new process which is a copy of the one that calls it, while exec() replaces the current process image with another (different) one.
- Both parent and child processes are executed simultaneously in case of fork() while Control never returns to the original program unless there is an exec() error.

Wait() system call

The **wait()** system call suspends execution of the calling process until one of its children terminates.

```
wait(int &status);
```

waitpid() system call :

By using this system call it is possible for parent process to synchronize its execution with a specific child process. Kernel will block the execution of a process that calls waitpid() system call if a child with specified pid of that process is in existence. It returns immediately when the child has terminated by returning termination status of the child.

```
int waitpid( int pid, int *status, int options);
```

nice() System Call:

Using nice() system call, we can change the priority of the process in multi-tasking system. The new priority number is added to the already existing value.

```
int nice(int inc);
```

nice() adds *inc* to the nice value. A higher nice value means a lower priority. The range of the nice value is +19 (low priority) to -20 (high priority).

```

#include<stdio.h>
main()
{
    int pid, retnice;

```

```

printf("press DEL to stop process \n");
pid=fork();
for(;;)
{
if(pid == 0)
{
retnice = nice (-5);
print("child gets higher CPU priority %d \n",
retnice);sleep(1);
}
else
{
retnice=nice(4);
print("Parent gets lower CPU priority %d \n",
retnice);sleep(1);
}
}
}

```

Orphan process

The child processes whose parent process has completed execution or terminated are called orphan process. Usually, a parent process waits for its child to terminate or finish its job but if parent fails to do so its child results in the Orphan process. In most cases, the Orphan process is immediately adopted by the init process (a very first process of the system).

Practical Assignments:

Set A

- (1) Implement the C Program to create a child process using fork(), display parent and child process id. Child process will display the message "I am Child Process" and the parent process should display "I am Parent Process".
- (2) Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

Set B

- (1) Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort. Both parent and child display the name of sorting method used and the data in sorted order.
- (2) Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid()).

Set C

- (1) Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of execve() system call. The child process uses execve() system call to load new program that uses this sorted array for performing the binary search to search the particular item in the array.
- (2) Implement the C Program to create a child process using fork(), Using exec() system call, child process will execute the program specified in Set A(1) and parent will continue by printing message "I am parent ".

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Signature of the instructor: _____

Date:

Assignment No.2: Operations on Processes

What is Shell?

Shell is an interface between user and operating system. It is the **command interpreter**, which accept the command name (program name) from user and executes that command/program. Shell mostly accepts the commands given by user from keyboard. Shell gets started automatically when Operating system is successfully started.

When shell is started successfully it generally display some prompt (such as #,\$ etc) to indicate the user that it is ready to accept and execute the command.

Shell executes the commands either **synchronously** or **asynchronously**.

When shell accepts the command then it locates the program file for that command, start its execution, wait for the program associated with the command to complete its execution and then display the prompt again to accept further command. This is called as Synchronous execution of shell.

In asynchronous execution shell accept the command from user, start the execution of program associated to the given command but does not wait for that program to finish its execution, display prompt to accept next command.

How Shell Execute the command?

1. Accept the command from user.
2. Tokenize the different parts of command.
3. First part given on the given command line is always a command name.
4. **Creates (Forks)** a child process for executing the program associated with given command.
5. Once child process is created successfully then it **loads (exec)** the binary (executable) image of given program in child process area.
6. Once the child process is loaded with given program it will start its execution while shell is **waiting (wait)** for it (child) to complete the execution. Shell will wait until child finish its execution.
7. Once child finish the execution then Shell wakeup, display the command prompt again and accept the command and continue.

Example

```
$ cat f1.dat f2.dat f3.dat
```

This command line has four tokens- cat, f1.dat, f2.dat and f3.dat. First token is the command name which is to be executed. In Linux operating system, some commands are internally coded and implemented by shell such as mkdir, rmdir, cd, pwd, ls, cat, grep, who etc.

Objective of this practical assignment is to simulate the shell which interprets all internal or predefined Linux commands and additionally implement to interpret following extended commands.

1. **count** : To count and display the number of lines, words and characters in a given file.
2. **typeline**: It will display the all or number of lines in given file.
3. **List**: It will list the files in current directory with some details of files
4. **Search**: It will allow searching the file for the occurrence of given string/pattern

Program Logic

1. Main function will execute and display the command prompt as \$
2. Accept the command at \$ prompt from user.
3. Separate or tokenize the different parts of command line.
4. Check that first part is one of the extended commands or not (count, typeline, list, search).
5. If the command is extended command then call corresponding functions which is implementing that command
6. Otherwise fork a new process and then load (exec) a program in that newly created process and execute it. Make the shell to wait until command finish its execution.
7. Display the prompt and continue until given command is "q" to Quit.

Practical Assignments:

Set A

Write a C program that behaves like a shell which displays the command prompt '**myshell\$**'. It accepts the command, tokenize the command line and execute it by creating the child process.

Also implement the additional command 'count' as

myshell\$ count c filename: It will display the number of characters in given file

myshell\$ count w filename: It will display the number of words in given file

myshell\$ count l filename: It will display the number of lines in given file

Set B

Write a C program that behaves like a shell which displays the command prompt '**myshell\$**'. It accepts the command, tokenize the command line and execute it by creating the child process.

Also implement the additional command 'list' as

myshell\$ list f dirname: It will display filenames in a given directory.

myshell\$ list n dirname: It will count the number of entries in a given directory.

myshell\$ list i dirname: It will display filenames and their inode number for the files in a given directory.

Set C

1. Write a C program that behaves like a shell which displays the command prompt '**myshell\$**'. It accepts the command, tokenize the command line and execute it by creating the child process.

Also implement the additional command 'typeline' as

myshell\$ typeline n filename: It will display first n lines of the file.

myshell\$ typeline -n filename: It will display last n lines of the file.

myshell\$ typeline a filename: It will display all the lines of the file.

2. Write a C program that behaves like a shell which displays the command prompt '**myshell\$**'. It accepts the command, tokenize the command line and execute it by creating the child process.

Also implement the additional command 'search' as

myshell\$ search f filename pattern : It will search the first occurrence of pattern in the given file

myshell\$ search a filename pattern : It will search all the occurrence of pattern in the given file

myshell\$ search c filename pattern : It will count the number of occurrence of pattern in the given file

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____/____/____

Assignment 3 : CPU Scheduling

CPU Scheduling is one of the important tasks performed by the operating system. In multiprogramming operating systems many processes are loaded into memory for execution and these processes are sharing the CPU and other resources of computer system.

Scheduler is a program that decides which program will execute next at the CPU or which program will be loaded into memory for execution.

CPU scheduler is a program module of an operating system that selects the process to execute next at CPU out of the processes that are in memory. This scheduler is also called as Short term scheduler or CPU Scheduler

The main objective is to increase system performance in accordance with the chosen set of criteria.

There are 3 types of schedulers as

- 1) Short Term Scheduler**
- 2) Long Term Scheduler**
- 3) Medium Term Scheduler**

We have to implement various Short Term Scheduling algorithms as a part of this practical assignment.

Various CPU Scheduling algorithms are:

- 1) First Come First Serve (FCFS)**
- 2) Shortest Job First (SJF)**
- 3) Priority Scheduling**
- 4) Round Robin Scheduling (RR)**

These scheduling algorithms are further classified into 2 types as **Preemptive** and **Non-Preemptive**. FCFS scheduling is always Non-Preemptive while Round Robin is always Preemptive, while Shortest Job First and Priority Scheduling can be preemptive or non-preemptive.

The performance of various scheduling algorithms is compared on the basis of following measures

- 1) CPU Utilization**
- 2) Throughput**
- 3) Turnaround Time**
- 4) Waiting Time**
- 5) Response Time**

Data Structures

To simulate the working of various CPU scheduling algorithms following data structures are required.

1) Ready Queue - It represents the queue of processes which ready to execute but waiting for CPU to become available. Scheduling algorithm will select appropriate process from the ready queue and dispatch it for execution. For FCFS and Round Robin this queue is strictly operated as **First In First out queue**. But for Shortest Job First and Priority Scheduling it will be operated as **Priority Queue**.

2) Process Control Block- It will maintain various details about the each process as processID, CPU-Burst, Arrival time, waiting time, completion time, execution time, turnaround time, etc. A structure can be used to define all these fields for processes and we can use array of structures of size n. (n is number of processes)

1) First Come First Serve Scheduling (FCFS):

In this algorithm the processes are executed in the same order in which they enter the ready queue. This algorithm is simple to implement but its performance may be worst many times.

DESCRIPTION:

The processes are allocated CPU in the order in which they enter in the ready queue and the processes free the CPU only after finishing the CPU burst. To calculate the average waiting time using the FCFS algorithm first the waiting time of the first process is kept zero and the waiting time of the second process is the burst time of the first process and the waiting time of the third process is the sum of the burst times of the first and the second process and so on. After calculating all the waiting times the average waiting time is calculated as the average of all the waiting times. FCFS mainly said that first come first serve the algorithm which came first will be served first.

Different variables required to be used in scheduling Algorithms are

at – arrival time

wt - Waiting time

tat - Turnaround time

tatavg - Average Turnaround time

wtavg - Average waiting time

qt - time quantum

bt - burst time

rembt – remaining burst time

t - Current time

ALGORITHM:

Step 1: Accept the number of processes in the ready Queue

Step 2: For each process in the ready Q, assign the process name and accept arrival time and burst time

Step 3: Set the waiting of the first process as 0 and its burst time as its turnaround time

$wt[0] = wtavg = 0;$

$tat[0] = tatavg = bt[0];$

Step 4: for each process in the Ready Q calculate

$wt[n] = wt[n-1] + bt[n-1] - at[n]$

$tat[n] = wt[n] + bt[n]$

Step 5: Calculate

$wtavg = \text{Total } wt / \text{Number of processes}$

$tatavg = \text{Total } tat / \text{Number of processes}$

Step 6: Stop the process

2) Shortest Job First (SJF):

In this algorithm the jobs will execute at the CPU according their next CPU-Burst time. The job in a ready queue which has shortest next CPU burst will execute next at the CPU. This algorithm can be preemptive or non-preemptive.

DESCRIPTION: (non-preemptive SJF)

To calculate the average waiting time in the shortest job first algorithm the sort the processes based on their burst time in ascending order then calculate the waiting time of each process as the sum of the burst times of all the process previous to that process.

ALGORITHM:

Step 1: Accept the number of processes in the ready Queue

Step 2: For each process in the ready Q, assign the process id and accept the arrival time and CPU burst time

$wt[0] = wtavg = 0;$

$tat[0] = tatavg = bt[0];$

Step 3: Arrange all the processes in sorted order of their arrival time and then burst time, from lowest to highest

Step 4: Set the waiting time of the first process in the sorted list as 0 and its turnaround time

as its bursttime.

Step 5: For each process in the sorted ready queue,
Calculate

$$wt[n] = wt[n-1] + bt[n-1] - at[n]$$

$$tat[n] = wt[n] + bt[n]$$

Step 6: Calculate

$$wtavg = \text{Total } wt / \text{Number of processes}$$

$$tatavg = \text{Total } tat / \text{Number of processes}$$

Step 7: Stop the process

3) Priority Scheduling (PS):

In this algorithm the processes will be executed according to their priority order. The process which has highest priority will execute first and the process which has least priority will be executed last by CPU. Priority scheduling can be preemptive or non-preemptive.

DESCRIPTION

To find out the order in which processes will be scheduled in the priority algorithm and to calculate the average waiting time and average turnaround time, sort the processes according to their priorities. The waiting time of each process is obtained by summing up the burst times of all the previous processes.

ALGORITHM:

Step 1: Accept the number of processes in the ready Queue

Step 2: For each process in the ready Q, assign the process id and accept the arrival time, CPU burst time and priority values

$$wt[0] = wtavg = 0;$$

$$tat[0] = tatavg = bt[0];$$

Step 3: Sort the ready queue in ascending order of the priority values.

Step 4: Set the waiting time of the first process in the sorted list as 0 and its turnaround time as its bursttime.

Step 5: For each process in the sorted ready queue,
Calculate

$$wt[n] = wt[n-1] + bt[n-1] - at[n]$$

$$tat[n] = wt[n] + bt[n]$$

Step 6: Calculate

$wtavg = \text{Total } wt / \text{Number of processes}$

$tatavg = \text{Total } tat / \text{Number of processes}$

Step 7: Stop the process

4) Round Robin Scheduling (RR):

This algorithm is mostly used in time-sharing operating systems like Unix/Linux. This algorithm gives the fair change of execution to each process in system in rotation. In this algorithm each process is allowed to execute for some fixed time quantum. If process has the CPU-burst more than time quantum then it will execute for the given time quantum. But if it has CPU-burst less than the time quantum then it will execute for its CPU-burst time and then immediately release the CPU so that it can be assigned to other process. The advantage of this algorithm is that the average waiting time is less. This algorithm use ready queue and is strictly operated as First In First Out queue. This algorithm is intrinsically preemptive scheduling algorithm.

DESCRIPTION:

Each process will be executed for a time quantum and if it finishes its CPU burst within the time quantum, it goes out of ready queue. But if it does not finish its CPU burst, it will go to the end of ready queue.

ALGORITHM:

Step 1: Accept the number of processes in the ready Queue and size of time quantum

Step 2: For each process in the ready Q, assign the process id and accept the arrival time and CPU burst time

$wt[0] = wtavg = 0;$

$tat[0] = tatavg = bt[0];$

Step 3: For each process i in the ready Q,

if burst time $[i] >$ time quantum

calculate cumulative $wt[i]$ and add the process to ready queue at the end

while adding the process at the end change $bt[i] = bt[i] - \text{time quantum}$

else

calculate cumulative $wt[i]$ and do not add the process to ready queue

Step 4: calculate turnaround time of all processes as

$tat[i] = \text{end time}[i] - at[i]$

Step 5: Calculate

$wtavg = \text{Total } wt / \text{Number of processes}$

$tatavg = \text{Total } tat / \text{Number of processes}$

Step 6: Stop the process

Set A:

- i. Write the program to simulate FCFS CPU-scheduling algorithm. Accept number of processes, n , from user. Take arrival time and CPU-burst time for n processes as input from the user. The output should be a Gantt chart, showing the scheduling of the processes according to FCFS. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.
- ii. Write the program to simulate Non-preemptive Shortest Job First (SJF) CPU-scheduling algorithm. Accept number of processes, n , from user. Take arrival time and CPU-burst time for n processes as input from the user. The output should be a Gantt chart, showing the scheduling of the processes according to Non-preemptive SJF. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor**Date of Completion** ____ / ____ / ____**Set B:**

- i. Write the program to simulate Preemptive Shortest Job First (SJF) CPU-scheduling algorithm. Accept number of processes, n , from user. Take arrival time and CPU-burst time for n processes as input from the user. The output should be a Gantt chart, showing the scheduling of the processes according to Preemptive SJF. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.
- ii. Write the program to simulate Non-preemptive Priority CPU-scheduling algorithm. Accept number of processes, n , from user. Take arrival time and CPU-burst time for n processes as input from the user. The output should be a Gantt chart, showing

the scheduling of the processes according to Non-preemptive Priority scheduling algorithm. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____ / ____ / ____

Set C:

- i. Write the program to simulate Preemptive Priority CPU-scheduling algorithm. Accept number of processes, n, from user. Take arrival time and CPU-burst time for n processes as input from the user. The output should be a Gantt chart, showing the scheduling of the processes according to Preemptive Priority Scheduling algorithm. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.
- ii. Write the program to simulate Round Robin (RR) scheduling algorithm. Accept number of processes, n, from user. Take arrival time and CPU-burst time for n processes and time quantum as input from the user. The output should be a Gantt chart, showing the scheduling of the processes according to Round Robin Scheduling algorithm. Also, the program should calculate waiting time, turnaround time for each process, and average waiting time and average turnaround time.

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____ / ____ / ____

Assignment No. 4: Demand Paging

Demand paging is one of the most commonly used methods of implementing the **Virtual Memory Management scheme**. Virtual memory management scheme has the following **Advantages**:

- 1) System can execute the process of larger size than the size of available physical memory.
- 2) Reduced disk IO operations.
- 3) More numbers of processes can be loaded into memory.
- 4) Performance of multi-programming and multi-tasking can be improved.

In this memory management scheme instead of loading entire process into memory, only required portion of each process is loaded in memory for execution. Different parts of physical memory of each process are brought into memory as and when required hence the name is demand paging.

Each process is logically divided into number of pages. Each page is of same size. Physical memory (RAM) is also divided into number of equal size frames. Size of a frame and a page is same. When process is to be loaded into memory its pages are loaded into available free frames.

Memory management scheme maintains the list of free frames.

It also maintains the page table for each process. Page table keeps track of various frames allocated to each page of process in physical memory. That is page table is used to map the logical pages of a process to frames in physical memory.

Memory Management Unit (MMU) of computer system is responsible to convert the logical address in process to physical memory in frame.

In demand paging memory management scheme no page of any process is brought into memory until it is not really required. Whenever page is required (demanded) then only it is brought into memory.

Page Fault:

When process requests some page during execution and that page is not available in physical memory then it is called as Page Fault.

Whenever page fault occurs Operating System checks really it is a page fault or it is an invalid memory reference. If page fault has occurred then system tries to load the corresponding page in available free frame.

Page Replacement:

When page fault occurs, system try to load the corresponding page into one of the available free frame. If no free frame is available then system try to replace one of the existing page in frame with new page. This is called as Page Replacement. Frame selected for page replacement is referred as victim frame. Page replacement algorithm of demand paging memory management scheme will decide which frame will be selected as victim frame. There are various page replacement algorithms as:

1. First In First Out Page Replacement (FIFO)
2. Optimal Page Replacement (OPT)
3. Least Recently Used Page Replacement (LRU)
4. Most Recently Used Page Replacement (MRU)
5. Most Frequently Used Page Replacement (MFU)
6. Least Frequently Used Page Replacement (LFU)
7. Second Change Page Replacement

Input to Page Replacement Algorithm:

1. **Reference String:** It is the list of numbers which represent the various page numbers demanded by the process.
2. **Number of Frames:** It represents the number of frames in physical memory.

Ex. Reference String: 3, 6, 5, 7, 3, 5, 2, 5, 7, 3, 2, 4, 2, 8, 3, 6

It means first process request the page number 3 then page number 6 then 5 and so on.

Data Structure:

1. Array of memory frames which is used to maintain which page is loaded in which frame. With each frame we may associate the counter or a time value whenever page is loaded in that frame.
2. Array of reference String.
3. Page Fault Count.

Output:

The output for page replacement algorithm is the contents of each page after every page reference.

Finally it should display the total number of page faults.

FIFO:

- In this page replacement algorithm the pages are replaced in the same order in which they are loaded in memory.
- We can associate a simple counter value with each frame when a page is loaded in memory.
- When page is to be replaced, select that page which has least counter value.
- It is most simple page replacement algorithm.

OPT:

- This algorithm looks into the future page demand of a process.
- Whenever page is to be replaced it will replace that page from the physical memory which will not be required for longest time.
- To implement this algorithm whenever page is to be replaced, we compare the pages in physical memory with their future occurrences in reference string. The page in physical memory which will not be required for longest period of time will be selected as victim.

LRU:

- This algorithm replaces that page from physical memory which is used least recently.
- To implement this algorithm we associate a next counter value or timer value with each frame in physical memory wherever it is loaded or referenced. When page replacement is to be performed, it will replace that frame in physical memory which has smallest counter value.

MRU:

- This algorithm replaces that page from physical memory which is used most recently.
- To implement this algorithm we associate a next counter value or timer value with each frame in physical memory wherever it is loaded or referenced. When page replacement is to be performed, it will replace that frame in physical memory which has greatest counter value.

MFU:

- This algorithm replaces that page from physical memory which is used most frequently.
- To implement this algorithm we need to keep track of how many times each page is used (frequency count). So, whenever a page is loaded in physical memory the

frequency count is set to 1 and it is incremented by 1 whenever page is referred. When page replacement is to be performed, it will replace that frame in physical memory which has highest frequency count.

- If frequency count is same for two or more pages then it will apply FCFS.

LFU:

- This algorithm replaces that page from physical memory which is used least frequently.
- To implement this algorithm we need to keep track of how many times each page is used (frequency count). So, whenever a page is loaded in physical memory the frequency count is set to 1 and it is incremented by 1 whenever page is referred. When page replacement is to be performed, it will replace that frame in physical memory which has smallest frequency count.
- If frequency count is the same for two or more pages then it will apply FCFS.

Second Chance Page Replacement:

- This algorithm is also called as Clock replacement policy.
- In this algorithm frames from physical memory are considered for the replacement in round robin manner. A page that has been accessed recently will get second chance and it will stay in memory for more time duration. If a page gets referred very frequently, it will not be removed from memory.
- This algorithm is an extension of FIFO page replacement.
- To implement this algorithm we have to associate second chance bit to each frame in physical memory. Initially when a page is loaded in memory its second chance bit is set to 0. Each time a memory frame is referenced set the second chance bit to 1. When page replacement is to be performed access the memory frames in round robin manner. If second chance bit is 1 then reset it to 0. If second chance bit is zero then replace the page.
- This algorithm gives far better performance than FCFS page replacement.

Set A

1. Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string. Give input n as the number of memory frames.

Reference String : 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8

- 1) Implement FIFO
- 2) Implement LRU

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____ / ____ / ____

Set B:

- I. Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string. Give input n as the number of memory frames.

Reference String : 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8

- 1) Implement OPT
- 2) Implement MFU

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____ / ____ / ____

Set C:

- I. Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string. Give input n as the number of memory frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

- 1) Implement MRU
- 2) Implement Second Chance Page Replacement.
- 3) Least Frequently Used

Assignment Evaluation

0: Not Done	[]	1: Incomplete	[]	2: Late Complete	[]
3: Needs Improvement	[]	4: Complete	[]	5: Well Done	[]

Signature of the Instructor

Date of Completion ____ / ____ / ____